

ΛΕΙΤΟΥΡΙΚΑ ΣΥΣΤΗΜΑΤΑ

2^Η ΕΡΓΑΣΤΗΡΙΑΚΗ ΑΣΚΗΣΗ (Περσινή)

Γιάννης Δρίβας ΑΜ: 5216

Περίληψη

Στην άσκηση αυτή θα υλοποιήσουμε ένα αρχείο καταγραφής (journal) για το σύστημα αρχείων FAT το οποίο θα κρατάει τις αλλαγές που θα κάνουμε στις βασικές δομές του συστήματός αρχείων μας μέσω κλήσεων εφαρμογών της βιβλιοθήκης lkl όπως για παράδειγμα η cftops. Η καταγραφή εστιάζει σε αλλαγές δομών όπως inodes, clusters, dentries όπου θα αποθηκεύονται στο πραγματικό μας σύστημα στο μονοπάτι /tmp/journal.txt

Περιγραφή της LKL Βιβλιοθήκης

Η lkl είναι μια βιβλιοθήκη του linux kernel η οποία μας επιτρέπει να χρησιμοποιήσουμε κλήσεις συστήματος και να επεξεργαστούμε drivers από το user space δηλαδή ένα χώρο διευθύνσεων κατανεμημένο για όλες τις διεργασίες που δεν σχετίζονται με το kernel. Ειδικότερα η lkl μας προσφέρει:

- Ανεξαρτησία από τις αλλαγές του πυρήνα του συστήματος καθιστώντας τις εφαρμογές συμβατές
- Βελτίωση στην απόδοση καθώς μας παρέχει πρόσβαση σε sys_calls χωρίς κατανάλωση επιπλέον πόρων αλλά και επειδή μπορούμε να παρέμβουμε στη προτεραιότητα των κλήσεων.
- Ασφάλεια μέσω του ελέγχου και προστασίας των κλήσεων του συστήματος .

Περιγραφή του FAT

Μια από τις υλοποιήσεις που περιέχει η LKL είναι οι οδηγοί για αρχεία που χρησιμοποιούνται για την επικοινωνία του συστήματος με άλλες συσκευές (usb , external hard drive) όταν τις συνδέουμε (mount).

Επιπλέον τα συστήματα Unix παρέχουν και τη λειτουργία του VFS το οποίο έχει ως βασική χρήση το abstraction των υλοποιήσεων της LKL έτσι ώστε να υπάρχει συμβατότητα μεταξύ των συστημάτων αρχείων και να μπορούν να χρησιμοποιηθούν οι ίδιες μέθοδοι ανεξαρτήτως της εφαρμογής.

Πιο συγκεκριμένα θα ασχοληθούμε με τα αρχεία FAT τα οποία χωρίζονται σε τρεις κατηγορίες:

- FAT 12
- FAT 16
- FAT 32

Οι διαφορές μεταξύ τους είναι ότι το καθένα είναι για πιο προηγμένες εκδόσεις λειτουργικών συστημάτων και ο αριθμός αναπαριστά τα bit καταχωρήσεων στον πίνακα αρχείων.

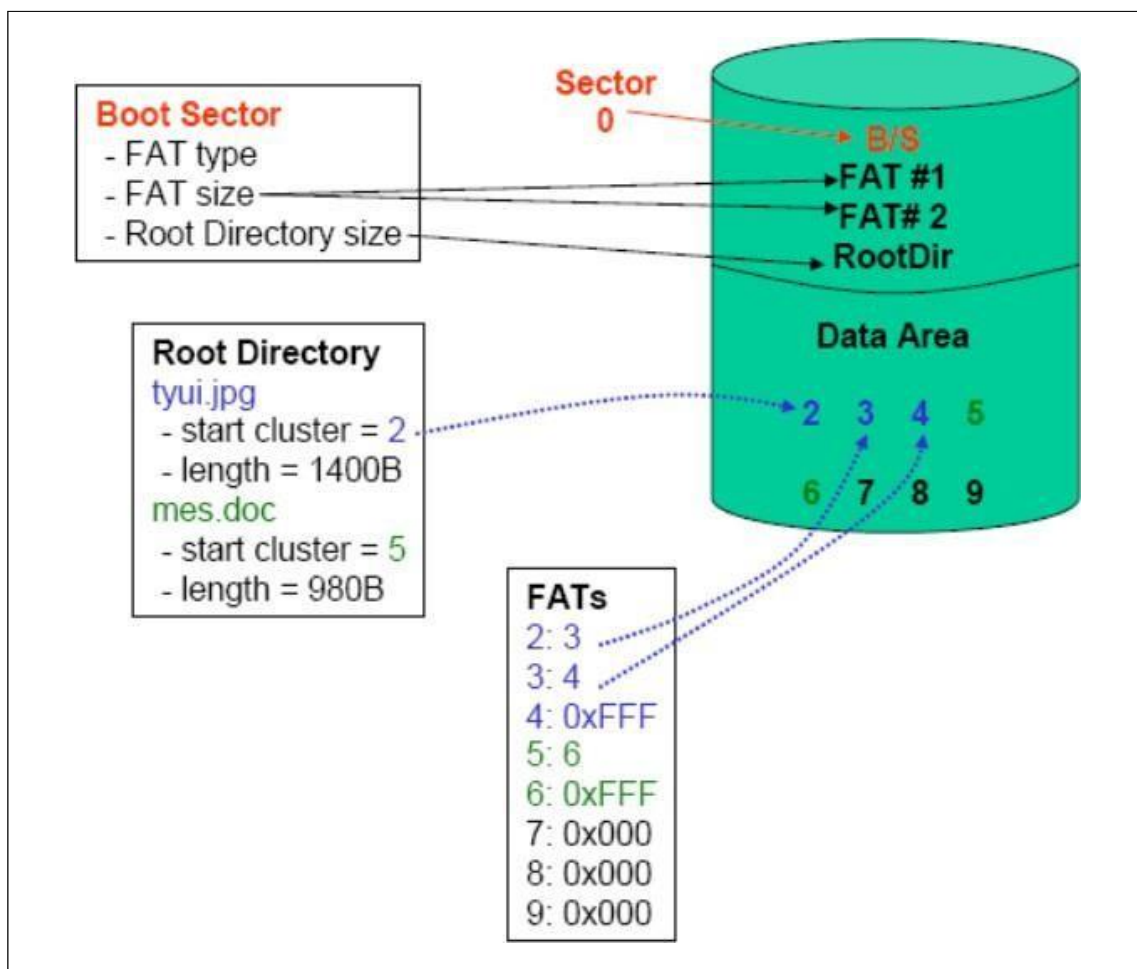
Σημείωση: Τα αρχεία FAT32 δεν υποστηρίζουν κάποια χαρακτηριστικά όπως κρυπτογράφηση των αρχείων, αρχεία μεγαλύτερα των 4GB, καταγραφή αρχείων (**journaling**) και κλπ.

Δομή του FAT

Οι δομή του FAT έχει ως εξής:

- **Boot Area:** Είναι η πρώτη δομή που βλέπει το σύστημα στην εκκίνηση και περιέχει πληροφορίες σχετικά με το σύστημα αρχείων, το μέγεθος των cluster την θέση του root κλπ.
- **File Allocation Tables (FATs):** Αυτή η δομή δίνει το όνομα του αρχείου και ευθύνεται να κρατάει αντίγραφο του FAT σε κάθε δίσκο για θέματα ασφαλείας. Είναι ουσιαστικά ένας πίνακας που κρατάει τις πληροφορίες σχετικά με τα clusters στον δίσκο.

- **Root Directory:** Περιέχει για όλα τα αρχεία και τους καταλόγους εγγραφές στον δίσκο με στοιχεία όπως το μέγεθος το όνομα την αρχική θέση του στον πίνακα fat κτλπ.
- **Data Region:** Περιέχει τα πραγματικά δεδομένα των αρχείων και καταλόγων τα οποία είναι υπό μορφή μπλοκ (clusters). Έτσι τα δεδομένα αποθηκεύονται διασκορπισμένα στο δίσκο σε πολλά clusters και η θέση του καθενός ελέγχεται από τις εγγραφές του FAT.



Δομή του VFS

Η χρησιμότητα του VFS που συναντούμε στο Linux είναι πολύ σημαντική καθώς μας επιτρέπει την ανάγνωση διαφορετικών συστημάτων αρχείων χρησιμοποιώντας ένα ενιαίο interface.

Η δομή του VFS έχει ως εξής:

- **Superblock:** Περιέχει πληροφορίες για ένα σύστημα αρχείων το οποίο δημιουργείται κατά τη διάρκεια διαμόρφωσης του συστήματος αρχείου.
- **Inode:** Απευθύνεται σε συγκεκριμένο αρχείο ή κατάλογο και κατέχει πληροφορίες του όπως δικαιώματα πρόσβασης, πότε τροποποιήθηκε το αρχείο, που βρίσκονται τα δεδομένα στον δίσκο κτλπ.
- **Dentry:** Δημιουργεί μια διασύνδεση μεταξύ του ονόματος ενός στοιχείου (αρχείο ή κατάλογος) με το αντίστοιχο Inode του
- **File:** Περιέχει πληροφορίες για ένα ανοικτό αρχείο όπως η τρέχουσα θέση της κεφαλής ανάγνωσης/εγγραφής κτλπ.
- **Address Space Object:** Περιέχει το χώρο διευθύνσεων ενός αρχείου στη μνήμη επιτρέποντας στο λειτουργικό σύστημα να διαχειρίζεται την ανάγνωση/εγγραφή των στοιχείων στη μνήμη/δίσκο.

Περιγραφή cptofs

Παρακάτω θα αναλύσουμε στοιχεία του κώδικα του αρχείου cptofs.c το οποίο βρίσκεται στον κατάλογο /lkl-source/tools/lkl:

- | |
|--|
| • lkl_sys_open() : Η συνάρτηση αυτή ανοίγει τον πυρήνα lkl για να χειριστεί το αρχείο |
| • lkl_sys_fchown() : Η συνάρτηση αυτή θέτει δικαιώματα στο αρχείο |

• lkl_sys_fsetxattr(): Η συνάρτηση αυτή θέτει ένα attribute προστασίας (cla.selinux) για το ποιος έχει δικαιώματα
• lkl_sys_read(): Η συνάρτηση αυτή διαβάζει από ένα buffer με τον πυρήνα της lkl
• lkl_sys_write(): Η συνάρτηση αυτή μέσω του πυρήνα της lkl γράφει σε ένα αρχείο
• lkl_sys_close(): Η συνάρτηση αυτή κλείνει το αρχείο που χειρίζεται ο πυρήνας της lkl
• lkl_sys_lstat(): Η συνάρτηση αυτή αναθέτει τιμές για τα στατιστικά όπως το file type, size κτλπ.
• lkl_sys_mkdir(): Η συνάρτηση αυτή μέσω του πυρήνα της lkl δημιουργεί ένα νέο κατάλογο
• lkl_sys_readlink(): Η συνάρτηση αυτή διαβάζει ένα Link με τον πυρήνα της lkl
• lkl_sys_symlink(): Η συνάρτηση αυτή απασχολεί τον πυρήνα της lkl για συμβολικό link (link το οποίο κατευθύνει σε άλλο αρχείο ή κατάλογο)
• lkl_sys_mount(): Η συνάρτηση αυτή κάνει το mounting του δίσκου στον πυρήνα της lkl
• lkl_sys_nanosleep(): Η συνάρτηση αυτή προσθέτει καθυστέρηση στον πυρήνα της lkl
• lkl_sys_halt(): Η συνάρτηση σταματά τον πυρήνα της lkl

Τοποθέτηση printk() στις βασικές δομές του FAT

1. struct super_operations fat_sops στο αρχείο fs/fat/inode.c:

- | |
|---|
| • .alloc_inode = fat_alloc_inode: Δημιουργεί νέο inode στο fat |
| • .free_inode = fat_free_inode: Ελευθερώνει ένα inode στο fat |

• .write_inode = fat_write_inode: Γράφει τη κατάσταση του inode στον δίσκο
• .evict_inode = fat_evict_inode: Καθαρίζει από το cache ένα inode
• .put_super = fat_put_super: Καταστρέφει ένα superblock
• .statfs = fat_statfs: Μας δείχνει στατιστικά του αρχείου(π.χ συνολικό αριθμό inodes)
• .remount_fs= fat_remount: Κάνει την επανασύνδεση του συστήματος αρχείου
• .show_options = fat_show_options: Μας δείχνει τις τρέχουσες επιλογές του αρχείου συστήματος

Για τα παραπάνω παραθέτουμε screenshot με τις προσθήκες τις printk():

```
static struct inode *fat_alloc_inode(struct super_block *sb)
{
    struct msdos_inode_info *ei;
    printk(KERN_INFO "Executing now: fat_alloc_inode");
    ei = alloc_inode_sb(sb, fat_inode_cache, GFP_NOFS);
    if (!ei)
        return NULL;

    init_rwsem(&ei->truncate_lock);
    /* Zeroing to allow iput() even if partial initialized inode. */
    ei->mmu_private = 0;
    ei->i_start = 0;
    ei->i_logstart = 0;
    ei->i_attrs = 0;
    ei->i_pos = 0;
    ei->i_crttime.tv_sec = 0;
    ei->i_crttime.tv_nsec = 0;

    return &ei->vfs_inode;
}
```

```
static void fat_free_inode(struct inode *inode)
{
    printk(KERN_INFO "Executing now: fat_free_inode");
    kmem_cache_free(fat_inode_cache, MSDOS_I(inode));
}
```

```

static int fat_write_inode(struct inode *inode, struct writeback_control *wbc)
{
    int err;
    printk(KERN_INFO "Executing now: fat_write_inode");
    if (inode->i_ino == MSDOS_FSINFO_INO) {
        struct super_block *sb = inode->i_sb;

        mutex_lock(&MSDOS_SB(sb)->s_lock);
        err = fat_clusters_flush(sb);
        mutex_unlock(&MSDOS_SB(sb)->s_lock);
    } else
        err = __fat_write_inode(inode, wbc->sync_mode == WB_SYNC_ALL);

    return err;
}

```

```

static void fat_evict_inode(struct inode *inode)
{
    printk(KERN_INFO "Executing now: fat_evict_inode");
    truncate_inode_pages_final(&inode->i_data);
    if (!inode->i_nlink) {
        inode->i_size = 0;
        fat_truncate_blocks(inode, 0);
    } else
        fat_free_eofblocks(inode);

    invalidate_inode_buffers(inode);
    clear_inode(inode);
    fat_cache_inval_inode(inode);
    fat_detach(inode);
}

```

```

static void fat_put_super(struct super_block *sb)
{
    struct msdos_sb_info *sbi = MSDOS_SB(sb);
    printk(KERN_INFO "Executing now: fat_put_super");
    fat_set_state(sb, 0, 0);

    iput(sbi->fsinfo_inode);
    iput(sbi->fat_inode);

    call_rcu(&sbi->rcu, delayed_free);
}

```

```

static int fat_statfs(struct dentry *dentry, struct kstatfs *buf)
{
    struct super_block *sb = dentry->d_sb;
    struct msdos_sb_info *sbi = MSDOS_SB(sb);
    u64 id = huge_encode_dev(sb->s_bdev->bd_dev);
    printk(KERN_INFO "Executing now: fat_statfs");
    /* If the count of free cluster is still unknown, counts it here. */
    if (sbi->free_clusters == -1 || !sbi->free_clus_valid) {
        int err = fat_count_free_clusters(dentry->d_sb);
        if (err)
            return err;
    }

    buf->f_type = dentry->d_sb->s_magic;
    buf->f_bsize = sbi->cluster_size;
    buf->f_blocks = sbi->max_cluster - FAT_START_ENT;
    buf->f_bfree = sbi->free_clusters;
    buf->f_bavail = sbi->free_clusters;
    buf->f_fsid = u64_to_fsid(id);
    buf->f_namelen =
        (sbi->options.isvfat ? FAT_LFN_LEN : 12) * NLS_MAX_CHARSET_SIZE;

    return 0;
}

```

```
static int fat_remount(struct super_block *sb, int *flags, char *data)
{
    bool new_ronly;
    struct msdos_sb_info *sbi = MSDOS_SB(sb);
    *flags |= SB_NODIRATIME | (sbi->options.isvfat ? 0 : SB_NOATIME);
    printk(KERN_INFO "Executing now: fat_statfs");
    sync_filesystem(sb);

    /* make sure we update state on remount. */
    new_ronly = *flags & SB_RDONLY;
    if (new_ronly != sb_ronly(sb)) {
        if (new_ronly)
            fat_set_state(sb, 0, 0);
        else
            fat_set_state(sb, 1, 1);
    }
    return 0;
}
```

```
static int fat_show_options(struct seq_file *m, struct dentry *root)
{
    struct msdos_sb_info *sbi = MSDOS_SB(root->d_sb);
    struct fat_mount_options *opts = &sbi->options;
    int isvfat = opts->isvfat;
    printk(KERN_INFO "Executing now: fat_show_options");
    if (!uid_eq(opts->fs_uid, GLOBAL_ROOT_UID))
        seq_printf(m, "uid=%u",
                   from_kuid_munged(&init_user_ns, opts->fs_uid));
    if (!gid_eq(opts->fs_gid, GLOBAL_ROOT_GID))
        seq_printf(m, "gid=%u",
                   from_kgid_munged(&init_user_ns, opts->fs_gid));
    seq_printf(m, "fmask=%04o", opts->fs_fmask);
    seq_printf(m, "dmask=%04o", opts->fs_dmask);
    if (opts->allow_utime)
        seq_printf(m, "allow_utime=%04o", opts->allow_utime);
    if (sbi->nls_disk)
        /* strip "cp" prefix from displayed option */
        seq_printf(m, "codepage=%s", &sbi->nls_disk->charset[2]);
    if (isvfat) {
        if (sbi->nls_io)
            seq_printf(m, "iocharset=%s", sbi->nls_io->charset);

        switch (opts->shortname) {
            case VFAT_SFN_DISPLAY_WIN95 | VFAT_SFN_CREATE_WIN95:
                seq_puts(m, "shortname=win95");
                break;
            case VFAT_SFN_DISPLAY_WINNT | VFAT_SFN_CREATE_WINNT:
                seq_puts(m, "shortname=winnt");
                break;
            case VFAT_SFN_DISPLAY_WINNT | VFAT_SFN_CREATE_WIN95:
                seq_puts(m, "shortname=mixed");
                break;
            case VFAT_SFN_DISPLAY_LOWER | VFAT_SFN_CREATE_WIN95:
                seq_puts(m, "shortname=lower");
                break;
            default:
                seq_puts(m, "shortname=unknown");
                break;
        }
    }
}
```

2. struct address_space_operations στο αρχείο fs/fat/inode.c:

- **.dirty_folio = block_dirty_folio:** Δείχνει πως ένα φύλλο στο χώρο διευθύνσεων έχει τροποποιηθεί
- **.invalidate_folio = block_invalidate_folio:** Δείχνει πως ένα φύλλο στο χώρο διευθύνσεων δεν είναι πλέον έγκυρο
- **.read_folio = fat_read_folio:** Διαβάζει δεδομένα από το αρχείο στη μνήμη

• .readahead = fat_readahead: Αυτή η συνάρτηση διαβάζει δεδομένα από την κρυφή μνήμη
• .writepage = fat_writepage: Γράφει μια σελίδα από το αρχείο στη μνήμη
• .writepages= fat_writepages: Γράφει πολλές σελίδες από το αρχείο στη μνήμη
• .write_begin = fat_write_begin: Ξεκινάει μια διεργασία εγγραφής στη μνήμη
• .write_end = fat_write_end: Τερματίζει μια διεργασία εγγραφής στη μνήμη
• .direct_IO = fat_direct_IO: Οι εφαρμογές παρακάμπτουν της cache εκτελώντας απευθείας τα I/O.
• .bmap = _fat_bmap: Μετατρέπει ένα λογικό block number αρχείου σε ένα φυσικό για τον δίσκο

Για τα παραπάνω παραθέτουμε screenshot με τις προσθήκες τις printk():

```
static int fat_writepage(struct page *page, struct writeback_control *wbc)
{
    printk(KERNEL_INFO "Executing now: fat_writepage");
    return block_write_full_page(page, fat_get_block, wbc);
}

static int fat_writepages(struct address_space *mapping,
                          struct writeback_control *wbc)
{
    printk(KERNEL_INFO "Executing now: fat_writepages");
    return mpage_writepages(mapping, wbc, fat_get_block);
}

static int fat_read_folio(struct file *file, struct folio *folio)
{
    printk(KERNEL_INFO "Executing now: fat_read_folio");
    return mpage_read_folio(folio, fat_get_block);
}

static void fat_readahead(struct readahead_control *rac)
{
    printk(KERNEL_INFO "Executing now: fat_read_head");
    mpage_readahead(rac, fat_get_block);
}
```

```

static int fat_write_begin(struct file *file, struct address_space *mapping,
                          loff_t pos, unsigned len,
                          struct page **pagep, void **fsdata)
{
    int err;

    *pagep = NULL;
    printk(KERN_INFO "Executing now: fat_write_begin");
    err = cont_write_begin(file, mapping, pos, len,
                          pagep, fsdata, fat_get_block,
                          &MSDOS_I(mapping->host)->mmu_private);
    if (err < 0)
        fat_write_failed(mapping, pos + len);
    return err;
}

static int fat_write_end(struct file *file, struct address_space *mapping,
                        loff_t pos, unsigned len, unsigned copied,
                        struct page *pagep, void *fsdata)
{
    struct inode *inode = mapping->host;
    int err;
    err = generic_write_end(file, mapping, pos, len, copied, pagep, fsdata);
    printk(KERN_INFO "Executing now: fat_write_end");
    if (err < len)
        fat_write_failed(mapping, pos + len);
    if (!(err < 0) && !(MSDOS_I(inode)->i_attrs & ATTR_ARCH)) {
        fat_truncate_time(inode, NULL, S_CTIME|S_MTIME);
        MSDOS_I(inode)->i_attrs |= ATTR_ARCH;
        mark_inode_dirty(inode);
    }
    return err;
}

```

```

static ssize_t fat_direct_IO(struct kiocb *iocb, struct iov_iter *iter)
{
    struct file *file = iocb->ki_filp;
    struct address_space *mapping = file->f_mapping;
    struct inode *inode = mapping->host;
    size_t count = iov_iter_count(iter);
    loff_t offset = iocb->ki_pos;
    ssize_t ret;
    printk(KERN_INFO "Executing now: fat_direct_IO");
    if (iov_iter_rw(iter) == WRITE) {
        /*
         * FIXME: blockdev_direct_IO() doesn't use ->write_begin(),
         * so we need to update the ->mmu_private to block boundary.
         *
         * But we must fill the remaining area or hole by nul for
         * updating ->mmu_private.
         *
         * Return 0, and fallback to normal buffered write.
         */
        loff_t size = offset + count;
        if (MSDOS_I(inode)->mmu_private < size)
            return 0;
    }

    /*
     * FAT need to use the DIO_LOCKING for avoiding the race
     * condition of fat_get_block() and ->truncate().
     */
    ret = blockdev_direct_IO(iocb, inode, iter, fat_get_block);
    if (ret < 0 && iov_iter_rw(iter) == WRITE)
        fat_write_failed(mapping, offset + count);

    return ret;
}

```

```

static sector_t _fat_bmap(struct address_space *mapping, sector_t block)
{
    sector_t blocknr;
    printk(KERN_INFO "Executing now: fat_bmap");
    /* fat_get_cluster() assumes the requested blocknr isn't truncated. */
    down_read(&MSDOS_I(mapping->host)->truncate_lock);
    blocknr = generic_block_bmap(mapping, block, fat_get_block_bmap);
    up_read(&MSDOS_I(mapping->host)->truncate_lock);

    return blocknr;
}

```

3. **struct fatent_operations fat12/16/32_ops** στο αρχείο fs/fat/fatent.c (ότι ισχύει στο fat12 είναι ίδιο και για τα 16/32):

• .ent_blocknr = fat12_ent_blocknr: Επιστρέφει τον αριθμό του Block που βρίσκεται μια εγγραφή FAT
• .ent_set_ptr = fat12_ent_set_ptr: Ορίζει ένα δείκτη στην εγγραφή
• .ent_bread = fat12_ent_bread: Κάνει ανάγνωση ενός Block από το δίσκο για μια συγκεκριμένη εγγραφή FAT
• .ent_get = fat12_ent_get: Επιστρέφει τη τιμή που δείχνει ο δείκτης στην εγγραφή
• .ent_put = fat12_ent_put: Ορίζει μία τιμή μιας εγγραφής FAT
• .ent_next = fat12_ent_next: Προχωράει τον δείκτη στην επόμενη εγγραφή FAT

Για τα παραπάνω παραθέτουμε screenshot με τις προσθήκες τις printk():

```

static void fat12_ent_blocknr(struct super_block *sb, int entry,
                             int *offset, sector_t *blocknr)
{
    struct msdos_sb_info *sbi = MSDOS_SB(sb);
    int bytes = entry + (entry >> 1);
    printk(KERN_INFO "Executing now: fat12_ent_blocknr");
    WARN_ON(!fat_valid_entry(sbi, entry));
    *offset = bytes & (sb->s_blocksize - 1);
    *blocknr = sbi->fat_start + (bytes >> sb->s_blocksize_bits);
}

static void fat_ent_blocknr(struct super_block *sb, int entry,
                            int *offset, sector_t *blocknr)
{
    struct msdos_sb_info *sbi = MSDOS_SB(sb);
    int bytes = (entry << sbi->fatent_shift);
    printk(KERN_INFO "Executing now: fat_ent_blocknr");
    WARN_ON(!fat_valid_entry(sbi, entry));
    *offset = bytes & (sb->s_blocksize - 1);
    *blocknr = sbi->fat_start + (bytes >> sb->s_blocksize_bits);
}

static void fat12_ent_set_ptr(struct fat_entry *fatent, int offset)
{
    struct buffer_head **bhs = fatent->bhs;
    printk(KERN_INFO "Executing now: fat12_ent_set_ptr");
    if (fatent->nr_bhs == 1) {
        WARN_ON(offset >= (bhs[0]->b_size - 1));
        fatent->u.ent12_p[0] = bhs[0]->b_data + offset;
        fatent->u.ent12_p[1] = bhs[0]->b_data + (offset + 1);
    } else {
        WARN_ON(offset != (bhs[0]->b_size - 1));
        fatent->u.ent12_p[0] = bhs[0]->b_data + offset;
        fatent->u.ent12_p[1] = bhs[1]->b_data;
    }
}

static void fat16_ent_set_ptr(struct fat_entry *fatent, int offset)
{
    printk(KERN_INFO "Executing now: fat16_ent_set_ptr");
    WARN_ON(offset & (2 - 1));
    fatent->u.ent16_p = (__le16 *) (fatent->bhs[0]->b_data + offset);
}

static void fat32_ent_set_ptr(struct fat_entry *fatent, int offset)
{
    printk(KERN_INFO "Executing now: fat16_ent_set_ptr");
    WARN_ON(offset & (4 - 1));
    fatent->u.ent32_p = (__le32 *) (fatent->bhs[0]->b_data + offset);
}

```

```

static int fat12_ent_bread(struct super_block *sb, struct fat_entry *fatent,
                          int offset, sector_t blocknr)
{
    struct buffer_head **bhs = fatent->bhs;
    printk(KERN_INFO "Executing now: fat12_ent_bread");
    WARN_ON(blocknr < MSDOS_SB(sb)->fat_start);
    fatent->fat_inode = MSDOS_SB(sb)->fat_inode;

    bhs[0] = sb_bread(sb, blocknr);
    if (!bhs[0])
        goto err;

    if ((offset + 1) < sb->s_blocksize)
        fatent->nr_bhs = 1;
    else {
        /* This entry is block boundary, it needs the next block */
        blocknr++;
        bhs[1] = sb_bread(sb, blocknr);
        if (!bhs[1])
            goto err_brelse;
        fatent->nr_bhs = 2;
    }
    fat12_ent_set_ptr(fatent, offset);
    return 0;
err_brelse:
    brelse(bhs[0]);
err:
    fat_msg_ratelimit(sb, KERN_ERR, "FAT read failed (blocknr %llu)",
                     (llu)blocknr);
    return -EIO;
}

static int fat_ent_bread(struct super_block *sb, struct fat_entry *fatent,
                        int offset, sector_t blocknr)
{
    const struct fatent_operations *ops = MSDOS_SB(sb)->fatent_ops;
    printk(KERN_INFO "Executing now: fat_ent_bread");
    WARN_ON(blocknr < MSDOS_SB(sb)->fat_start);
    fatent->fat_inode = MSDOS_SB(sb)->fat_inode;
    fatent->bhs[0] = sb_bread(sb, blocknr);
    if (!fatent->bhs[0]) {
        fat_msg_ratelimit(sb, KERN_ERR, "FAT read failed (blocknr %llu)",
                         (llu)blocknr);
        return -EIO;
    }
    fatent->nr_bhs = 1;
    ops->ent_set_ptr(fatent, offset);
    return 0;
}

```

```

static int fat12_ent_get(struct fat_entry *fatent)
{
    u8 **ent12_p = fatent->u.ent12_p;
    int next;
    printk(KERN_INFO "Executing now: fat12_ent_get");
    spin_lock(&fat12_entry_lock);
    if (fatent->entry & 1)
        next = (*ent12_p[0] >> 4) | (*ent12_p[1] << 4);
    else
        next = (*ent12_p[1] << 8) | *ent12_p[0];
    spin_unlock(&fat12_entry_lock);

    next &= 0x0fff;
    if (next >= BAD_FAT12)
        next = FAT_ENT_EOF;
    return next;
}

static int fat16_ent_get(struct fat_entry *fatent)
{
    int next = le16_to_cpu(*fatent->u.ent16_p);
    printk(KERN_INFO "Executing now: fat16_ent_bread");
    WARN_ON((unsigned long)fatent->u.ent16_p & (2 - 1));
    if (next >= BAD_FAT16)
        next = FAT_ENT_EOF;
    return next;
}

static int fat32_ent_get(struct fat_entry *fatent)
{
    int next = le32_to_cpu(*fatent->u.ent32_p) & 0xffffffff;
    printk(KERN_INFO "Executing now: fat32_ent_get");
    WARN_ON((unsigned long)fatent->u.ent32_p & (4 - 1));
    if (next >= BAD_FAT32)
        next = FAT_ENT_EOF;
    return next;
}

```

```

static void fat12_ent_put(struct fat_entry *fatent, int new)
{
    u8 **ent12_p = fatent->u.ent12_p;
    printk(KERN_INFO "Executing now: fat12_ent_put");
    if (new == FAT_ENT_EOF)
        new = EOF_FAT12;

    spin_lock(&fat12_entry_lock);
    if (fatent->entry & 1){
        *ent12_p[0] = (new << 4) | (*ent12_p[0] & 0x0f);
        *ent12_p[1] = new >> 4;
    } else {
        *ent12_p[0] = new & 0xff;
        *ent12_p[1] = (*ent12_p[1] & 0xf0) | (new >> 8);
    }
    spin_unlock(&fat12_entry_lock);

    mark_buffer_dirty_inode(fatent->bhs[0], fatent->fat_inode);
    if (fatent->nr_bhs == 2)
        mark_buffer_dirty_inode(fatent->bhs[1], fatent->fat_inode);
}

static void fat16_ent_put(struct fat_entry *fatent, int new)
{
    printk(KERN_INFO "Executing now: fat16_ent_put");
    if (new == FAT_ENT_EOF)
        new = EOF_FAT16;

    *fatent->u.ent16_p = cpu_to_le16(new);
    mark_buffer_dirty_inode(fatent->bhs[0], fatent->fat_inode);
}

static void fat32_ent_put(struct fat_entry *fatent, int new)
{
    printk(KERN_INFO "Executing now: fat32_ent_put");
    WARN_ON(new & 0xf0000000);
    new |= le32_to_cpu(*fatent->u.ent32_p) & ~0xffffffff;
    *fatent->u.ent32_p = cpu_to_le32(new);
    mark_buffer_dirty_inode(fatent->bhs[0], fatent->fat_inode);
}

```

```

static int fat12_ent_next(struct fat_entry *fatent)
{
    u8 **ent12_p = fatent->u.ent12_p;
    struct buffer_head **bhs = fatent->bhs;
    u8 *nextp = ent12_p[1] + 1 + (fatent->entry & 1);
    printk(KERN_INFO "Executing now: fat12_ent_next");
    fatent->entry++;
    if (fatent->nr_bhs == 1) {
        WARN_ON(ent12_p[0] > (u8 *) (bhs[0]->b_data +
                                     (bhs[0]->b_size - 2)));
        WARN_ON(ent12_p[1] > (u8 *) (bhs[0]->b_data +
                                     (bhs[0]->b_size - 1)));
        if (nextp < (u8 *) (bhs[0]->b_data + (bhs[0]->b_size - 1))) {
            ent12_p[0] = nextp - 1;
            ent12_p[1] = nextp;
            return 1;
        }
    } else {
        WARN_ON(ent12_p[0] != (u8 *) (bhs[0]->b_data +
                                     (bhs[0]->b_size - 1)));
        WARN_ON(ent12_p[1] != (u8 *) bhs[1]->b_data);
        ent12_p[0] = nextp - 1;
        ent12_p[1] = nextp;
        brelse(bhs[0]);
        bhs[0] = bhs[1];
        fatent->nr_bhs = 1;
        return 1;
    }
    ent12_p[0] = NULL;
    ent12_p[1] = NULL;
    return 0;
}

static int fat16_ent_next(struct fat_entry *fatent)
{
    const struct buffer_head *bh = fatent->bhs[0];
    printk(KERN_INFO "Executing now: fat16_ent_next");
    fatent->entry++;
    if (fatent->u.ent16_p < ((__le16 *) (bh->b_data + (bh->b_size - 2)))) {
        fatent->u.ent16_p++;
        return 1;
    }
    fatent->u.ent16_p = NULL;
    return 0;
}

static int fat32_ent_next(struct fat_entry *fatent)
{
    const struct buffer_head *bh = fatent->bhs[0];
    printk(KERN_INFO "Executing now: fat32_ent_next");
    fatent->entry++;
    if (fatent->u.ent32_p < ((__le32 *) (bh->b_data + (bh->b_size - 4)))) {
        fatent->u.ent32_p++;
        return 1;
    }
}

```

4. struct file_operations στο αρχείο fs/fat/file.c:

- | |
|---|
| • .llseek = generic_file_llseek: Ελέγχει την θέση ανάγνωσης/εγγραφής του αρχείου |
| • .read_iter = generic_file_read_iter: Χειρίζεται την ανάγνωση δεδομένων του αρχείου |
| • .write_iter = generic_file_write_iter: Χειρίζεται την εγγραφή δεδομένων του αρχείου |
| • .mmap = generic_file_mmap: Χειρίζεται την λειτουργία μνήμης του αρχείου |
| • .release = fat_file_release: Κλείνει και ελευθερώνει του δεσμευμένους πόρους του αρχείου |

- | |
|---|
| <ul style="list-style-type: none"> • .unlocked_ioctl = fat_generic_ioctl: Χειρίζεται τα ειδικά αιτήματα ελέγχου του αρχείου (IOCTLs) |
| <ul style="list-style-type: none"> • .compat_ioctl = compat_ptr_ioctl: Χειρίζεται τα ειδικά αιτήματα ελέγχου παλιών αρχείων (IOCTLs) |
| <ul style="list-style-type: none"> • .fsync = fat_file_fsync: Χειρίζεται τον συγχρονισμό του αρχείου έτσι ώστε όλες οι αλλαγές του στο δίσκο να έχουν διεκπεραιωθεί |
| <ul style="list-style-type: none"> • .splice_read = generic_file_splice_read: Χειρίζεται την αντιγραφή και επικόλληση των δεδομένων από ένα αρχείο σε ένα άλλο |
| <ul style="list-style-type: none"> • .fallocate = fat_fallocate: Χειρίζεται την δέσμευση χώρου του αρχείου στον δίσκο |

Για τα παραπάνω παραθέτουμε screenshot με τις προσθήκες τις `printk()`:

Στο αρχείο `fs/read_write.c`:

```

//
loff_t generic_file_llseek(struct file *file, loff_t offset, int whence)
{
    struct inode *inode = file->f_mapping->host;
    printk(KERN_INFO "Executing now: generic_file_llseek");
    return generic_file_llseek_size(file, offset, whence,
                                     inode->i_sb->s_maxbytes,
                                     i_size_read(inode));
}

```

Στο αρχείο `mm/filemap.c`:

```

generic_file_read_iter(struct kiocb *iocb, struct iov_iter *iter)
{
    size_t count = iov_iter_count(iter);
    ssize_t retval = 0;
    printk(KERN_INFO "Executing now: generic_file_read_iter");
    if (!count)
        return 0; /* skip atime */

    if (iocb->ki_flags & IOCB_DIRECT) {
        struct file *file = iocb->ki_filp;
        struct address_space *mapping = file->f_mapping;
        struct inode *inode = mapping->host;

        if (iocb->ki_flags & IOCB_NOWAIT) {
            if (filemap_range_needs_writeback(mapping, iocb->ki_pos,
                                                iocb->ki_pos + count - 1))
                return -EAGAIN;
        } else {
            retval = filemap_write_and_wait_range(mapping,
                                                    iocb->ki_pos,
                                                    iocb->ki_pos + count - 1);

            if (retval < 0)
                return retval;
        }
    }
}

```

```

ssize_t __generic_file_write_iter(struct kiocb *iocb, struct iov_iter *from)
{
    struct file *file = iocb->ki_filp;
    struct address_space *mapping = file->f_mapping;
    struct inode *inode = mapping->host;
    ssize_t written = 0;
    ssize_t err;
    ssize_t status;
    printk(KERN_INFO "Executing now: generic_file_write_iter\n");
    /* We can write back this queue in page reclaim */
    current->backing_dev_info = inode_to_bdi(inode);
    err = file_remove_privs(file);
    if (err)
        goto out;

    err = file_update_time(file);
    if (err)
        goto out;
}

```

```

int generic_file_mmap(struct file *file, struct vm_area_struct *vma)
{
    struct address_space *mapping = file->f_mapping;
    printk(KERN_INFO "Executing now: \n");
    if (!mapping->a_ops->read_folio)
        return -ENOEXEC;
    file_accessed(file);
    vma->vm_ops = &generic_file_vm_ops;
    return 0;
}

```

```

int generic_file_mmap(struct file *file, struct vm_area_struct *vma)
{
    printk(KERN_INFO "Executing now: generic_file_mmap\n");
    return -ENOSYS;
}

```

Στο αρχείο fs/fat/file.c:

```

long fat_generic_ioctl(struct file *filp, unsigned int cmd, unsigned long arg)
{
    struct inode *inode = file_inode(filp);
    u32 __user *user_attr = (u32 __user *)arg;
    printk(KERN_INFO "Executing now: fat_generic_ioctl");
    switch (cmd) {
        case FAT_IOCTL_GET_ATTRIBUTES:
            return fat_ioctl_get_attributes(inode, user_attr);
        case FAT_IOCTL_SET_ATTRIBUTES:
            return fat_ioctl_set_attributes(filp, user_attr);
        case FAT_IOCTL_GET_VOLUME_ID:
            return fat_ioctl_get_volume_id(inode, user_attr);
        case FITRIM:
            return fat_ioctl_fitrim(inode, arg);
        default:
            return -ENOTTY; /* Inappropriate ioctl for device */
    }
}

static int fat_file_release(struct inode *inode, struct file *filp)
{
    printk(KERN_INFO "Executing now: fat_file_release");
    if ((filp->f_mode & FMODE_WRITE) &&
        MSDOS_SB(inode->i_sb)->options.flush) {
        fat_flush_inodes(inode->i_sb, inode, NULL);
        set_current_state(TASK_UNINTERRUPTIBLE);
        io_schedule_timeout(HZ/10);
    }
    return 0;
}

int fat_file_fsync(struct file *filp, loff_t start, loff_t end, int datasync)
{
    struct inode *inode = filp->f_mapping->host;
    int err;
    printk(KERN_INFO "Executing now: fat_file_fsync");
    err = __generic_file_fsync(filp, start, end, datasync);
    if (err)
        return err;

    err = sync_mapping_buffers(MSDOS_SB(inode->i_sb)->fat_inode->i_mapping);
    if (err)
        return err;

    return blkdev_issue_flush(inode->i_sb->s_bdev);
}

```

```

static long fat_fallocate(struct file *file, int mode,
                          loff_t offset, loff_t len)
{
    int nr_cluster; /* Number of clusters to be allocated */
    loff_t mm_bytes; /* Number of bytes to be allocated for file */
    loff_t ondisksize; /* block aligned on-disk size in bytes */
    struct inode *inode = file->f_mapping->host;
    struct super_block *sb = inode->i_sb;
    struct msdos_sb_info *sbi = MSDOS_SB(sb);
    int err = 0;
    printk(KERN_INFO "Executing now: fat_fallocate");
    /* No support for hole punch or other fallocate flags. */
    if (mode & ~FALLOC_FL_KEEP_SIZE)
        return -EOPNOTSUPP;

    /* No support for dir */
    if (!S_ISREG(inode->i_mode))
        return -EOPNOTSUPP;
}

```

Στο αρχείο fs/splice.c:

```

ssize_t generic_file_splice_read(struct file *in, loff_t *ppos,
                                struct pipe_inode_info *pipe, size_t len,
                                unsigned int flags)
{
    struct iov_iter to;
    struct kiocb kiocb;
    int ret;
    printk(KERN_INFO "Executing now: generic_file_splice_read");
    iov_iter_pipe(&to, READ, pipe, len);
    init_sync_kiocb(&kiocb, in);
    kiocb.ki_pos = *ppos;
    ret = call_read_iter(in, &kiocb, &to);
    if (ret > 0) {
        *ppos = kiocb.ki_pos;
        file_accessed(in);
    } else if (ret < 0) {
        /* free what was emitted */
        pipe_discard_from(pipe, to.start_head);
        /*
         * callers of ->splice_read() expect -EAGAIN on
         * "can't put anything in there", rather than -EFAULT.
         */
        if (ret == -EFAULT)
            ret = -EAGAIN;
    }
    return ret;
}

```

5. struct inode_operations στο αρχείο fs/fat/file.c:

- **.setattr = fat_setattr:** Χειρίζεται την ανάθεση των ιδιοτήτων ενός Inode
- **.getattr = fat_getattr:** Χειρίζεται την ανάκτηση των ιδιοτήτων ενός Inode

Για τα παραπάνω παραθέτουμε screenshot με τις προσθήκες τις `printk()`:

```

int fat_setattr(struct user_namespace *mnt_userns, struct dentry *dentry,
                struct iattr *attr)
{
    struct msdos_sb_info *sbi = MSDOS_SB(dentry->d_sb);
    struct inode *inode = d_inode(dentry);
    unsigned int ia_valid;
    int error;
    printk(KERN_INFO "Executing now: fat_setattr");
    /* Check for setting the inode time. */
    ia_valid = attr->ia_valid;
    if (ia_valid & TIMES_SET_FLAGS) {
        if (fat_allow_set_time(mnt_userns, sbi, inode))
            attr->ia_valid &= ~TIMES_SET_FLAGS;
    }

    error = setattr_prepare(mnt_userns, dentry, attr);
    attr->ia_valid = ia_valid;
    if (error) {
        if (sbi->options.quiet)
            error = 0;
        goto out;
    }

    /*
     * Expand the file. Since inode_setattr() updates ->i_size
     * before calling the ->truncate(), but FAT needs to fill the
     * hole before it. XXX: this is no longer true with new truncate

```

```

int fat_getattr(struct user_namespace *mnt_userns, const struct path *path,
               struct kstat *stat, u32 request_mask, unsigned int flags)
{
    struct inode *inode = d_inode(path->dentry);
    struct msdos_sb_info *sbi = MSDOS_SB(inode->i_sb);
    printk(KERNEL_INFO "Executing now: fat_getattr");
    generic_fillattr(mnt_userns, inode, stat);
    stat->blksize = sbi->cluster_size;

    if (sbi->options.nfs == FAT_NFS_NOSTALE_RO) {
        /* Use i_pos for ino. This is used as fileid of nfs. */
        stat->ino = fat_i_pos_read(sbi, inode);
    }

    if (sbi->options.isvfat && request_mask & STATX_BTIME) {
        stat->result_mask |= STATX_BTIME;
        stat->btime = MSDOS_I(inode)->i_crttime;
    }

    return 0;
}
EXPORT_SYMBOL_GPL(fat_getattr);

```

6. **struct inode_operations msdos_dir_inode_operations** του αρχείου `fs/fat/namei_msdos.c` και **struct inode_operations vfat_dir_inode_operations** του αρχείου `fs/fat/namei_vfat.c`:

• .create = vfat_create: Χειρίζεται τη δημιουργία ενός αρχείου σε ένα κατάλογο
• .lookup = vfat_lookup: Χειρίζεται τον εντοπισμό ενός αρχείου ή ενός καταλόγου σε έναν κατάλογο με βάση το όνομά του.
• unlink = vfat_unlink: Χειρίζεται την αφαίρεση ενός συνδέσμου από ένα αρχείο ή κατάλογο και εάν δεν υπάρχει άλλος σύνδεσμος τότε το στοιχείο διαγράφεται
• .mkdir = vfat_mkdir: Χειρίζεται τη δημιουργία ενός νέου καταλόγου σε έναν άλλο κατάλογο
• .rmdir = vfat_rmdir: Χειρίζεται την διαγραφή ενός καταλόγου που βρίσκεται σε έναν άλλο με τη προϋπόθεση ότι είναι κενός
• .rename = vfat_rename: Χειρίζεται τη μετονομασία ενός καταλόγου ή αρχείου
• .setattr = fat_setattr: Χειρίζεται την εγγραφή των ιδιοτήτων ενός Inode
• .getattr = fat_getattr: Χειρίζεται την ανάκτηση των ιδιοτήτων ενός Inode

Για τα παραπάνω παραθέτουμε screenshot με τις προσθήκες τις `printk()` στο αρχείο `namei_vfat.c`:

```
static int vfat_create(struct user_namespace *mnt_userns, struct inode *dir,
                      struct dentry *dentry, umode_t mode, bool excl)
{
    struct super_block *sb = dir->i_sb;
    struct inode *inode;
    struct fat_slot_info sinfo;
    struct timespec64 ts;
    int err;
    printk(KERN_INFO "Executing vfat_create");
    mutex_lock(&MSDOS_SB(sb)->s_lock);

    ts = current_time(dir);
    err = vfat_add_entry(dir, &dentry->d_name, 0, 0, &ts, &sinfo);
    if (err)
        goto out;
    inode_inc_iversion(dir);

    inode = fat_build_inode(sb, sinfo.de, sinfo.i_pos);
    brelse(sinfo.bh);
    if (IS_ERR(inode)) {
        err = PTR_ERR(inode);
        goto out;
    }
    inode_inc_iversion(inode);

    d_instantiate(dentry, inode);
out:
    mutex_unlock(&MSDOS_SB(sb)->s_lock);
    return err;
}

static int vfat_rmdir(struct inode *dir, struct dentry *dentry)
{
    struct inode *inode = d_inode(dentry);
    struct super_block *sb = dir->i_sb;
    struct fat_slot_info sinfo;
    int err;
    printk(KERN_INFO "Executing vfat_rmdir");
    mutex_lock(&MSDOS_SB(sb)->s_lock);

    err = fat_dir_empty(inode);
    if (err)
        goto out;
    err = vfat_find(dir, &dentry->d_name, &sinfo);
    if (err)
        goto out;

    err = fat_remove_entries(dir, &sinfo); /* and releases bh */
    if (err)
        goto out;
    drop_nlink(dir);

    clear_nlink(inode);
    fat_truncate_time(inode, NULL, S_ETIME|S_MTIME);
    fat_detach(inode);
}
```

```

static int vfat_unlink(struct inode *dir, struct dentry *dentry)
{
    struct inode *inode = d_inode(dentry);
    struct super_block *sb = dir->i_sb;
    struct fat_slot_info sinfo;
    int err;
    printk(KERNEL_INFO "Executing vfat_rmdir");
    mutex_lock(&MSDOS_SB(sb)->s_lock);

    err = vfat_find(dir, &dentry->d_name, &sinfo);
    if (err)
        goto out;

    err = fat_remove_entries(dir, &sinfo); /* and releases bh */
    if (err)
        goto out;
    clear_nlink(inode);
    fat_truncate_time(inode, NULL, S_ETIME|S_MTIME);
    fat_detach(inode);
    vfat_d_version_set(dentry, inode_query_iversion(dir));
out:
    mutex_unlock(&MSDOS_SB(sb)->s_lock);

    return err;
}

static int vfat_mkdir(struct user_namespace *mnt_userns, struct inode *dir,
                     struct dentry *dentry, umode_t mode)
{
    struct super_block *sb = dir->i_sb;
    struct inode *inode;
    struct fat_slot_info sinfo;
    struct timespec64 ts;
    int err, cluster;
    printk(KERNEL_INFO "Executing vfat_mkdir");
    mutex_lock(&MSDOS_SB(sb)->s_lock);

    ts = current_time(dir);
    cluster = fat_alloc_new_dir(dir, &ts);
    if (cluster < 0) {
        err = cluster;
        goto out;
    }
    err = vfat_add_entry(dir, &dentry->d_name, 1, cluster, &ts, &sinfo);
    if (err)
        goto out_free;
    inode_inc_iversion(dir);
    inc_nlink(dir);
}

```

```

static struct dentry *vfat_lookup(struct inode *dir, struct dentry *dentry,
                                unsigned int flags)
{
    struct super_block *sb = dir->i_sb;
    struct fat_slot_info sinfo;
    struct inode *inode;
    struct dentry *alias;
    int err;
    printk(KERNEL_INFO "Executing now: vfat_lookup");
    mutex_lock(&MSDOS_SB(sb)->s_lock);

    err = vfat_find(dir, &dentry->d_name, &sinfo);
    if (err) {
        if (err == -ENOENT) {
            inode = NULL;
            goto out;
        }
        goto error;
    }

    inode = fat_build_inode(sb, sinfo.de, sinfo.i_pos);
    brelse(sinfo.bh);
    if (IS_ERR(inode)) {
        err = PTR_ERR(inode);
        goto error;
    }
}

```

```

static int vfat_rename(struct inode *old_dir, struct dentry *old_dentry,
                      struct inode *new_dir, struct dentry *new_dentry)
{
    struct buffer_head *dotdot_bh;
    struct msdos_dir_entry *dotdot_de = NULL;
    struct inode *old_inode, *new_inode;
    struct fat_slot_info old_sinfo, sinfo;
    struct timespec64 ts;
    loff_t new_i_pos;
    int err, is_dir, corrupt = 0;
    struct super_block *sb = old_dir->i_sb;
    printk(KERN_INFO "Executing now: vfat_rename");
    old_sinfo.bh = sinfo.bh = dotdot_bh = NULL;
    old_inode = d_inode(old_dentry);
    new_inode = d_inode(new_dentry);
    mutex_lock(&MSDOS_SB(sb)->s_lock);
    err = vfat_find(old_dir, &old_dentry->d_name, &old_sinfo);
    if (err)
        goto out;

    if (old_dir != new_dir) {
        err = vfat_get_dotdot_de(old_inode, &dotdot_bh, &dotdot_de);
        if (err)
            goto out;
    }
}

```

Εκτέλεση cptofs

Στη διαδρομή /tools/lkl εκτελούμε την εντολή ./cptofs -p -i /tmp/disk -t vfat lklfuse.o / και παρατηρούμε πλέον τις Printk που εμφανίζονται:

```

0.011697] printk: console [lkl console0] enabled
0.011773] clocksource: jiffies: mask: 0xffffffff max_cycles: 0xffffffff, max_idle_ns: 19112604462750000 ns
0.011994] NET: Registered PF_NETLINK/PF_ROUTE protocol family
0.012210] lkl_pci: probe of lkl_pci failed with error -1
0.016677] vgaarb: loaded
0.017036] clocksource: Switched to clocksource lkl
0.017236] NET: Registered PF_INET protocol family
0.017649] IP idents hash table entries: 2048 (order: 2, 16384 bytes, linear)
0.017882] tcp_listen_portaddr_hash hash table entries: 512 (order: 0, 4096 bytes, linear)
0.017972] Table-perturb hash table entries: 65536 (order: 6, 262144 bytes, linear)
0.017996] TCP established hash table entries: 1024 (order: 1, 8192 bytes, linear)
0.018024] TCP bind hash table entries: 1024 (order: 2, 16384 bytes, linear)
0.018040] TCP: Hash tables configured (established 1024 bind 1024)
0.018086] UDP hash table entries: 128 (order: 0, 4096 bytes, linear)
0.018101] UDP-Lite hash table entries: 128 (order: 0, 4096 bytes, linear)
0.018227] PCI: CLS 0 bytes, default 32
0.018287] virtio-mmio: Registering device virtio-mmio.0 at 0x10000000-0x1000147, IRQ 1.
0.019722] workingset: timestamp_bits=62 max_order=15 bucket_order=0
0.020942] io scheduler mq-deadline registered
0.021041] io scheduler kyber registered
0.025854] virtio_blk virtio0: 1/0/0 default/read/poll queues
0.026423] virtio_blk virtio0: [vda] 614400 512-byte logical blocks (315 MB/300 MiB)
0.027915] vda:
0.030806] NET: Registered PF_INET6 protocol family
0.033572] Segment Routing with IPv6
0.033788] In-situ OAM (IOAM) with IPv6
0.033871] sit: IPv6, IPv4 and MPLS over IPv4 tunneling driver
0.035671] Warning: unable to open an initial console.
0.035854] This architecture does not have kernel memory protection.
0.035919] Run /init as init process
0.036015] Executing now: generic_file_read_iter
0.038222] Executing now: fat_alloc_inode
0.038386] Executing now: fat_alloc_inode
0.038401] Executing now: fat_alloc_inode
0.049126] Executing now: fat_statfs
0.049875] reboot: Restarting system

```

Τώρα αντιγράφουμε το αρχείο lklfuse.c που είναι μεγαλύτερο και παρατηρούμε πως έχουμε περισσότερα μηνύματα καθώς γίνονται σε περισσότερα δεδομένα αντιγραφή:

```
[ 0.082582] This architecture does not have kernel memory protection.
[ 0.082608] Run /init as init process
[ 0.082666] Executing now: generic_file_read_iter
[ 0.096730] Executing now: fat_alloc_inode
[ 0.096864] Executing now: fat_alloc_inode
[ 0.096883] Executing now: fat_alloc_inode
[ 0.098333] Executing now: vfat_lookup
[ 0.098478] Executing now: fat_alloc_inode
[ 0.098547] Executing now: fat_setattr
[ 0.098583] Executing now: fat_ent_blocknr
[ 0.098613] Executing now: fat_ent_bread
[ 0.098670] Executing now: fat16_ent_set_prt
[ 0.098684] Executing now: fat16_ent_bread
[ 0.098696] Executing now: fat16_ent_put
[ 0.098711] Executing now: fat_ent_blocknr
[ 0.098722] Executing now: fat16_ent_set_prt
[ 0.098793] Executing now: fat16_ent_bread
[ 0.098810] Executing now: fat16_ent_put
[ 0.098852] Executing now: generic_file_write_iter
[ 0.098872] Executing now: fat_write_begin
[ 0.098951] Executing now: fat_ent_blocknr
[ 0.098968] Executing now: fat_ent_bread
[ 0.098980] Executing now: fat16_ent_set_prt
[ 0.098992] Executing now: fat16_ent_bread
[ 0.099051] Executing now: fat16_ent_put
[ 0.099081] Executing now: fat_write_end
[ 0.099101] Executing now: generic_file_write_iter
[ 0.099114] Executing now: fat_write_begin
[ 0.099132] Executing now: fat_write_end
[ 0.099152] Executing now: generic_file_write_iter
[ 0.099218] Executing now: fat_write_begin
[ 0.099232] Executing now: fat_ent_blocknr
[ 0.099235] Executing now: fat_ent_bread
[ 0.099239] Executing now: fat16_ent_set_prt
[ 0.099242] Executing now: fat16_ent_bread
[ 0.099245] Executing now: fat16_ent_put
```

```
[ 0.099245] Executing now: fat16_ent_put
[ 0.099250] Executing now: fat_ent_blocknr
[ 0.099253] Executing now: fat_ent_bread
[ 0.099256] Executing now: fat16_ent_set_prt
[ 0.099260] Executing now: fat16_ent_bread
[ 0.099263] Executing now: fat_ent_blocknr
[ 0.099266] Executing now: fat_ent_bread
[ 0.099269] Executing now: fat16_ent_set_prt
[ 0.099272] Executing now: fat16_ent_bread
[ 0.099276] Executing now: fat16_ent_put
[ 0.099300] Executing now: fat_ent_blocknr
[ 0.099303] Executing now: fat_ent_bread
[ 0.099306] Executing now: fat16_ent_set_prt
[ 0.099310] Executing now: fat16_ent_bread
[ 0.099320] Executing now: fat_write_end
[ 0.099330] Executing now: generic_file_write_iter
[ 0.099392] Executing now: fat_write_begin
[ 0.099407] Executing now: fat_write_end
[ 0.099437] Executing now: fat_setattr
[ 0.100632] Executing now: fat_file_release
[ 1.100928] Executing now: fat_statfs
[ 1.101166] Executing now: fat_writepages
[ 1.101473] Executing now: fat_write_inode
[ 1.102703] reboot: Restarting system
```

Δημιουργία του journal

Inode.c

Για την δημιουργία του Journal θα χρησιμοποιήσουμε τις κλήσεις του host (open,close,write) όπου θα πρέπει να δηλωθούν ως extern στον κώδικά μας. Επιπλέον θα πρέπει να ορίσουμε έναν global file descriptor ο οποίος θα δημιουργηθεί στο αρχείο inode.c. Με αυτόν τον τρόπο ανοίγουμε το αρχείο journal δίνοντας και τα αντίστοιχα δικαιώματα κατά το Mount μέσω της κλήσης της συνάρτησης fat_fill_super και το κλείσιμο μέσω της fat_put_super. Το επόμενο βήμα είναι να καταγράψουμε τις κατάλληλες τιμές για το αρχείο μας. Στην συνάρτηση fat_fill_inode καταγράφουμε τη θέση το inode μέσω της μεταβλητής inode->i_no την πρώτη θέση και λογική θέση καθώς και το μέγεθός του μέσω των i_start, i_logstart, i_size. Στην fat_write_inode ποιο inode γράψαμε καθώς και το μέγεθός του μέσω των μεταβλητών inode->i_no, inode->i_size. Τέλος στην fat_fill_super καταγράφουμε τα ελεύθερα cluster και το προηγούμενο εισαγόμενο με τις μεταβλητές sbi->free_clusters και sbi->prev_free, το μέγεθος του cluster και το είδος του fat(12/16/32) μέσω των sbi->fat_bits, sbi->cluster_size αλλά και πληροφορίες για τη δομή του δίσκου μέσω των fat->start, fat_length. Παρακάτω φαίνεται και ο αντίστοιχος κώδικας:

```

/* DOS dates from 1980/1/1 through 2107/12/31 */
#define FAT_DATE_MIN (0<<9 | 1<<5 | 1)
#define FAT_DATE_MAX (127<<9 | 12<<5 | 31)
#define FAT_TIME_MAX (23<<11 | 59<<5 | 29)
int my_journal_fd = -1;
extern int open(const char *pathname, int flags, mode_t mode);
extern int close(int fd);
extern long write(int fd, const void *buf, unsigned long count);

```

```

//start
if (my_journal_fd < 0) {
    my_journal_fd = open("/tmp/journal.txt", O_CREAT | O_WRONLY | O_APPEND, 0666);
}

{
    char buffer[512];
    int length;
    length = scnprintf(buffer, sizeof(buffer),
        "JOURNAL - fat_fill_super: fat_bits=%d cluster_size=%u "
        "fat_start=%u fat_length=%lu free_clusters=%d prev_free=%u\n",
        sbi->fat_bits, sbi->cluster_size,
        sbi->fat_start, sbi->fat_length,
        sbi->free_clusters, sbi->prev_free);
    if (length > 0 && my_journal_fd >= 0) {
        write(my_journal_fd, buffer, length);
        printk(KERN_INFO "%s", buffer);
    }
}

//end

```

```

static void fat_put_super(struct super_block *sb)
{
    if (my_journal_fd >= 0) {
        close(my_journal_fd);
        my_journal_fd = -1;
    }
    struct msdos_sb_info *sbi = MSDOS_SB(sb);
    printk(KERN_INFO "Executing now: fat_put_super");
    fat_set_state(sb, 0, 0);

    iput(sbi->fsinfo_inode);
    iput(sbi->fat_inode);

    call_rcu(&sbi->rcu, delayed_free);
}

```

```

/* doesn't deal with root inode */
int fat_fill_inode(struct inode *inode, struct msdos_dir_entry *de)
{
    struct msdos_sb_info *sbi = MSDOS_SB(inode->i_sb);
    int error;
    //start
    char buffer[1024];
    int length;
    //end

    //start
    length = scnprintf(buffer, sizeof(buffer),
        "JOURNAL - fat_fill_inode: Inode %lu i_start=%d "
        "i_logstart=%d i_size=%lld \n",
        inode->i_ino,
        MSDOS_I(inode)->i_start,
        MSDOS_I(inode)->i_logstart,
        inode->i_size);
    if (length > 0 && my_journal_fd >= 0) {
        write(my_journal_fd, buffer, length);
        printk(KERN_INFO "%s", buffer);
    }
    //end

    return 0;
}

```

```

static int fat_write_inode(struct inode *inode, struct writeback_control *wbc)
{
    int err;
    //start
    //write on journal the inode number and its size
    char buffer[256];
    int length;
    length = scnprintf(buffer, sizeof(buffer),
        "JOURNAL - fat_write_inode: Wrote Inode %lu, Size (i_size): %lld bytes\n",
        inode->i_ino, inode->i_size);
    if (length > 0 && my_journal_fd >= 0) {
        write(my_journal_fd, buffer, length);
        printk(KERN_INFO "%s", buffer);
    } else {
        printk(KERN_INFO "Executing now: fat_write_inode\n");
    }
    //end
    if (inode->i_ino == MSDOS_FSINFO_INO) {
        struct super_block *sb = inode->i_sb;

        mutex_lock(&MSDOS_SB(sb)->s_lock);
        err = fat_clusters_flush(sb);
        mutex_unlock(&MSDOS_SB(sb)->s_lock);
    } else
        err = __fat_write_inode(inode, wbc->sync_mode == WB_SYNC_ALL);

    return err;
}

```

Dir.c

Στο συγκεκριμένο αρχείο περνάμε τον file descriptor και την write όπως κάναμε ως extern όπως κάναμε και στο inode.c και στην συνάρτηση fat_add_entries καταγράφουμε το inode του γονικού καταλόγου καθώς και πόσα νέα slots δεσμεύτηκαν στο directory μέσω των μεταβλητών dir->i_ino και dir->nr_slots. Αντίστοιχα στην fat_remove_entries καταγράφουμε πόσα slot αποδεσμεύτηκαν και από ποιο inode μέσω των ίδιων μεταβλητών. Παρακάτω φαίνεται και ο αντίστοιχος κώδικας:

```

#include <linux/slab.h>
#include <linux/compat.h>
#include <linux/uaccess.h>
#include <linux/iversion.h>
#include "fat.h"
extern int my_journal_fd;
extern long write(int fd, const void *buf, unsigned long count);

```

```

//start
//log the slots that were requested
{
    char buffer[256];
    int length;
    length = scnprintf(buffer, sizeof(buffer),
        "JOURNAL - fat_add_entries: Added %d entries in directory Inode %lu\n",
        sinfo->nr_slots, dir->i_ino);

    if (length > 0 && my_journal_fd >= 0) {
        write(my_journal_fd, buffer, length);
        printk(KERN_INFO "%s", buffer);
    }
}

return 0;

error:
    brelse(bh);
    for (i = 0; i < nr_bhs; i++)
        brelse(bhs[i]);
    return err;

error_remove:
    brelse(bh);
    if (free_slots)
        __fat_remove_entries(dir, pos, free_slots);
    return err;
}
EXPORT_SYMBOL_GPL(fat_add_entries);

```

```

//start
//log how many entries were removed from which directory
{
    char buffer[256];
    int length;
    length = scnprintf(buffer, sizeof(buffer),
        "JOURNAL - fat_remove_entries: Removed %d entries from directory Inode %lu\n",
        sinfo->nr_slots, dir->i_ino);

    if (length > 0 && my_journal_fd >= 0) {
        write(my_journal_fd, buffer, length);
        printk(KERN_INFO "%s", buffer);
    }
}

return 0;
}
EXPORT_SYMBOL_GPL(fat_remove_entries);

```

Fatent.c

Με παρόμοιο τρόπο όπως και πριν περνάμε το fd και την write ενώ στην συνάρτηση fat_alloc_clusters καταγράφουμε τα cluster που

δεσμεύει ο πυρήνας για δεδομένα μέσω της μεταβλητής entry. Τέλος στην fat_free_clusters καταγράφουμε τα cluster που αποδεσμεύονται μέσω της μεταβλητής cluster. Παρακάτω φαίνεται και ο κώδικας:

```
#include <linux/blkdev.h>
#include <linux/sched/signal.h>
#include <linux/backing-dev-defs.h>
#include "fat.h"
extern int my_journal_fd;
extern long write(int fd, const void *buf, unsigned long count);
```

```
//start
// log the allocated cluster and the remaining free clusters
{
    char buffer[256];
    int length;
    length = scnprintf(buffer, sizeof(buffer),
        "JOURNAL - fat_alloc_clusters: Inode %lu allocated cluster %d. Free clusters: %d\n",
        inode->i_ino, entry, sbi->free_clusters);
    if (length > 0 && my_journal_fd >= 0) {
        write(my_journal_fd, buffer, length);
        printk(KERN_INFO "%s", buffer);
    }
}
//end

idx_clus++;
if (idx_clus == nr_cluster)
    goto out;

/*
 * fat_collect_bhs() gets ref-count of bhs,
 * so we can still use the prev_ent.
 */
prev_ent = fatent;
}
count++;
```

```
//start
// log the freed cluster and the remaining free clusters
{
    char buffer[256];
    int length;
    length = scnprintf(buffer, sizeof(buffer),
        "JOURNAL - fat_free_clusters: Inode %lu freed cluster %d. Free clusters: %d\n",
        inode->i_ino, cluster, sbi->free_clusters);
    if (length > 0 && my_journal_fd >= 0) {
        write(my_journal_fd, buffer, length);
        printk(KERN_INFO "%s", buffer);
    }
}
//end

if (nr_bhs + fatent.nr_bhs > MAX_BUF_PER_PAGE) {
    if (sb->s_flags & SB_SYNCHRONOUS) {
        err = fat_sync_bhs(bhs, nr_bhs);
        if (err)
            goto error;
    }
    err = fat_mirror_bhs(sb, bhs, nr_bhs);
    if (err)
        goto error;
    for (i = 0; i < nr_bhs; i++)
        brelse(bhs[i]);
    nr_bhs = 0;
}
```

File.c

Στο αρχείο αυτό περνάμε πάλι τις παραμέτρους και στην συνάρτηση `fat_free` καταγράφουμε το πρώτο cluster και το πρώτο λογικό cluster:

```
#include <linux/capability.h>
#include <linux/module.h>
#include <linux/compat.h>
#include <linux/mount.h>
#include <linux/blkdev.h>
#include <linux/backing-dev.h>
#include <linux/fsnotify.h>
#include <linux/security.h>
#include <linux/falloc.h>
#include "fat.h"
extern int my_journal_fd;
extern long write(int fd, const void *buf, unsigned long count);
```

```
static int fat_free(struct inode *inode, int skip)
{
    struct super_block *sb = inode->i_sb;
    int err, wait, free_start, i_start, i_logstart;

    //start
    char buffer[1024];
    int length;
    //end

    if (MSDOS_I(inode)->i_start == 0)
        return 0;

    fat_cache_inval_inode(inode);

    wait = IS_DIRSYNC(inode);
    i_start = free_start = MSDOS_I(inode)->i_start;
    i_logstart = MSDOS_I(inode)->i_logstart;

    //start
    length = scnprintf(buffer, sizeof(buffer), "JOURNAL - fat_free: i_start: %d, i_logstart: %d\n", i_start, i_logstart);
    if (length > 0){
        write(my_journal_fd, buffer, length);
        printk(KERN_INFO "%s", buffer);
    }
    //end

    /* First, we write the new file size. */
```

Namei_vfat.c

Αφού αρχικοποιήσουμε και εδώ τις μεταβλητές στην συνάρτηση `vfat_create` καταγράφουμε το όνομα του αρχείου καταγραφής μας μέσω της μεταβλητής `dentry->d_name.name`. Στην `vfat_rmdir` καταγράφουμε ποιο directory διαγράφουμε, στην `vfat_mkdir` ποιο directory δημιουργούμε και τέλος στην `vfat_unlink` ποιο αρχείο διαγράφουμε. Τέλος στο struct του `vfat_lookup` αμέσως μετά την κλήση της `vfat_find` καταγράφουμε ποιο αρχείο βρήκαμε καθώς ψάχναμε καθώς και σε ποιο inode. Παρακάτω φαίνεται και ο κώδικας:

```

#include <linux/module.h>
#include <linux/ctype.h>
#include <linux/slab.h>
#include <linux/namei.h>
#include <linux/kernel.h>
#include <linux/iversion.h>
#include "fat.h"
extern int my_journal_fd;
extern long write(int fd, const void *buf, unsigned long count);

```

```

//start
{
    char buffer[256];
    int length;

    length = scnprintf(buffer, sizeof(buffer),
        "JOURNAL - vfat_create: Created file '%s' in directory Inode %lu\n",
        dentry->d_name.name, dir->i_ino);

    if (length > 0 && my_journal_fd >= 0) {
        write(my_journal_fd, buffer, length);
        printk(KERN_INFO "%s", buffer);
    }
}
//
out:
mutex_unlock(&MSDOS_SB(sb)->s_lock);
return err;
}

```

```

//start
{
    char buffer[256];
    int length;

    length = scnprintf(buffer, sizeof(buffer),
        "JOURNAL - vfat_rmdir: Removed directory '%s' from Inode %lu\n",
        dentry->d_name.name, dir->i_ino);

    if (length > 0 && my_journal_fd >= 0) {
        write(my_journal_fd, buffer, length);
        printk(KERN_INFO "%s", buffer);
    }
}
//end
out:
mutex_unlock(&MSDOS_SB(sb)->s_lock);

return err;
}

```

```

//start
{
    char buffer[256];
    int length;

    length = scnprintf(buffer, sizeof(buffer),
        "JOURNAL - vfat_unlink: Deleted file '%s' from Inode %lu\n",
        dentry->d_name.name, dir->i_ino);

    if (length > 0 && my_journal_fd >= 0) {
        write(my_journal_fd, buffer, length);
        printk(KERN_INFO "%s", buffer);
    }
}
//end

out:
mutex_unlock(&MSDOS_SB(sb)->s_lock);

return err;
}

//start
{
    char buffer[256];
    int length;

    length = scnprintf(buffer, sizeof(buffer),
        "JOURNAL - vfat_mkdir: Created directory '%s' in Inode %lu\n",
        dentry->d_name.name, dir->i_ino);

    if (length > 0 && my_journal_fd >= 0) {
        write(my_journal_fd, buffer, length);
        printk(KERN_INFO "%s", buffer);
    }
}
//end

return 0;

out_free:
fat_free_clusters(dir, cluster);
out:
mutex_unlock(&MSDOS_SB(sb)->s_lock);
return err;
}

```

```

static struct dentry *vfat_lookup(struct inode *dir, struct dentry *dentry,
                                unsigned int flags)
{
    struct super_block *sb = dir->i_sb;
    struct fat_slot_info sinfo;
    struct inode *inode;
    struct dentry *alias;
    int err;
    printk(KERN_INFO "Executing now: vfat_lookup");
    mutex_lock(&MSDOS_SB(sb)->s_lock);

    err = vfat_find(dir, &dentry->d_name, &sinfo);
    if (err) {
        if (err == -ENOENT) {
            inode = NULL;
            goto out;
        }
        goto error;
    }

    //start
    {
        char buffer[256];
        int length;
        length = scnprintf(buffer, sizeof(buffer),
                           "JOURNAL - vfat_lookup: Found '%.*s' in directory Inode %lu\n", dentry->d_name.len, dentry->d_name.name, dir->i_ino);
        if (length > 0 && my_journal_fd >= 0) {
            write(my_journal_fd, buffer, length);
            printk(KERN_INFO "%s", buffer);
        }
    }
    //end
}

```

Εκτέλεση των προγραμμάτων

Εκτελώντας άλλη μια φορά το crtofs.c έχουμε τα παρακάτω αποτελέσματα:

```
myy601@myy601lab2:~/lkl-source/tools/lkl$ ./cptofs -p -i /tmp/disk -t vfat lklfuse.c /
[ 0.000000] Linux version 6.1.0 (myy601@myy601lab2) (gcc (Debian 12.2.0-14) 12.2.0, GNU ld (GNU Binutils for Debian) 2.40) #21 Mon May 11 23:49:34 EEST 2026
[ 0.000000] memblock address range: 0x7facalc00000 - 0x7faca8000000
[ 0.000000] Zone ranges:
[ 0.000000]   Normal [mem 0x00007facalc00000-0x00007faca7ffffff]
[ 0.000000] Movable zone start for each node
[ 0.000000] Early memory node ranges
[ 0.000000]   node   0: [mem 0x00007facalc00000-0x00007faca7ffffff]
[ 0.000000] Initmem setup node 0 [mem 0x00007facalc00000-0x00007faca7ffffff]
[ 0.000000] Built 1 zonelists, mobility grouping on. Total pages: 25250
[ 0.000000] Kernel command line: mem=100M virtio_mmio.device=328@0x1000000:1
[ 0.000000] Dentry cache hash table entries: 16384 (order: 5, 131072 bytes, linear)
[ 0.000000] Inode-cache hash table entries: 8192 (order: 4, 65536 bytes, linear)
[ 0.000000] mem auto-init: stack:all(zero), heap alloc:off, heap free:off
[ 0.000000] Memory: 100768K/102400K available (3452K kernel code, 488K rdata, 775K rodata, 99K init, 313K bss, 1632K reserved, 0K cma-reserved)
[ 0.000000] SLUB: HWalign=32, Order=0-3, MinObjects=0, CPUs=1, Nodes=1
[ 0.000000] NR_IRQS: 4096
[ 0.000000] lkl: irq: initialized
[ 0.000000] clocksource: lkl: mask: 0xffffffffffffff max_cycles: 0x1cd42e4dffb, max_idle_ns: 881590591483 ns
[ 0.000000] lkl: time and timers initialized (irq2)
[ 0.000017] pid_max: default: 4096 minimum: 301
[ 0.000075] Mount-cache hash table entries: 512 (order: 0, 4096 bytes, linear)
[ 0.000076] Mountpoint-cache hash table entries: 512 (order: 0, 4096 bytes, linear)
[ 0.003041] printk: console [lkl_console0] enabled
[ 0.003072] clocksource: jiffies: mask: 0xffffffff max_cycles: 0xffffffff, max_idle_ns: 19112604462750000 ns
[ 0.003175] NET: Registered PF_INETLINK/PF_ROUTE protocol family
[ 0.003255] lkl_pci: probe of lkl_pci failed with error -1
[ 0.003940] vgaarb: loaded
[ 0.003977] clocksource: Switched to clocksource lkl
[ 0.004079] NET: Registered PF_INET protocol family
[ 0.004146] IP idents hash table entries: 2048 (order: 2, 16384 bytes, linear)
[ 0.004218] tcp_listen_portaddr hash hash table entries: 512 (order: 0, 4096 bytes, linear)
[ 0.004236] Table-perturb hash table entries: 65536 (order: 6, 262144 bytes, linear)
[ 0.004240] TCP established hash table entries: 1024 (order: 1, 8192 bytes, linear)
[ 0.004260] TCP bind hash table entries: 1024 (order: 2, 16384 bytes, linear)
[ 0.004292] TCP: Hash tables configured (established 1024 bind 1024)
[ 0.004359] UDP hash table entries: 128 (order: 0, 4096 bytes, linear)
[ 0.004383] UDP-Lite hash table entries: 128 (order: 0, 4096 bytes, linear)
[ 0.004411] PCI: CLS 0 bytes, default 32
[ 0.004469] virtio-mmio: Registering device virtio-mmio.0 at 0x1000000-0x1000147, IRQ 1.
[ 0.004854] workingset: timestamp bits=62 max_order=15 bucket_order=0
[ 0.005435] io scheduler mq-deadline registered
[ 0.005460] io scheduler kyber registered
[ 0.007390] virtio blk virtio0: 1/0/0 default/read/poll queues
[ 0.007390] virtio blk virtio0: 1/0/0 default/read/poll queues
[ 0.007472] virtio_blk virtio0: [vda] 614400 512-byte logical blocks (315 MB/300 MiB)
[ 0.007755] vda:
[ 0.008094] NET: Registered PF_INET6 protocol family
[ 0.008457] Segment Routing with IPv6
[ 0.008688] In-situ OAM (IOAM) with IPv6
[ 0.008719] sit: IPv6, IPv4 and MPLS over IPv4 tunneling driver
[ 0.009361] Warning: unable to open an initial console.
[ 0.009376] This architecture does not have kernel memory protection.
[ 0.009386] Run /init as init process
[ 0.009411] Executing now: generic_file_read_iter
[ 0.010228] Executing now: fat_alloc_inode
[ 0.010254] Executing now: fat_alloc_inode
[ 0.010266] Executing now: fat_alloc_inode
[ 0.010789] JOURNAL - fat fill super: fat bits=16 cluster_size=8192 fat_start=16 fat_length=160 free_clusters=-1 prev_free=2
[ 0.010929] Executing now: vfat_lookup
[ 0.010941] JOURNAL - vfat lookup: Found 'lklfuse.c' in directory Inode 1
[ 0.010978] Executing now: fat_alloc_inode
[ 0.010987] JOURNAL - fat fill inode: Inode 3 i_start=3 i_logstart=3 i_size=14192
[ 0.011018] Executing now: fat_setattr
[ 0.011025] JOURNAL - fat free: i_start: 3, i_logstart: 3
[ 0.011045] Executing now: fat_ent_blocknr
[ 0.011046] Executing now: fat_ent_bread
[ 0.011076] Executing now: fat16_ent_set_pt
[ 0.011086] Executing now: fat16_ent_bread
[ 0.011094] Executing now: fat16_ent_put
[ 0.011108] JOURNAL - fat free_clusters: Inode 3 freed cluster 4. Free clusters: -1
[ 0.011124] Executing now: fat_ent_blocknr
[ 0.011125] Executing now: fat16_ent_set_pt
[ 0.011132] Executing now: fat16_ent_bread
[ 0.011140] Executing now: fat16_ent_put
[ 0.011152] JOURNAL - fat_free_clusters: Inode 3 freed cluster 268435455. Free clusters: -1
[ 0.011175] Executing now: generic_file_write_iter
[ 0.011177] Executing now: fat_write_begin
[ 0.011187] Executing now: fat_ent_blocknr
[ 0.011195] Executing now: fat_ent_bread
[ 0.011203] Executing now: fat16_ent_set_pt
[ 0.011223] Executing now: fat16_ent_bread
[ 0.011233] Executing now: fat16_ent_put
[ 0.011247] JOURNAL - fat alloc_clusters: Inode 3 allocated cluster 3. Free clusters: -1
[ 0.011265] Executing now: fat_write_end
```

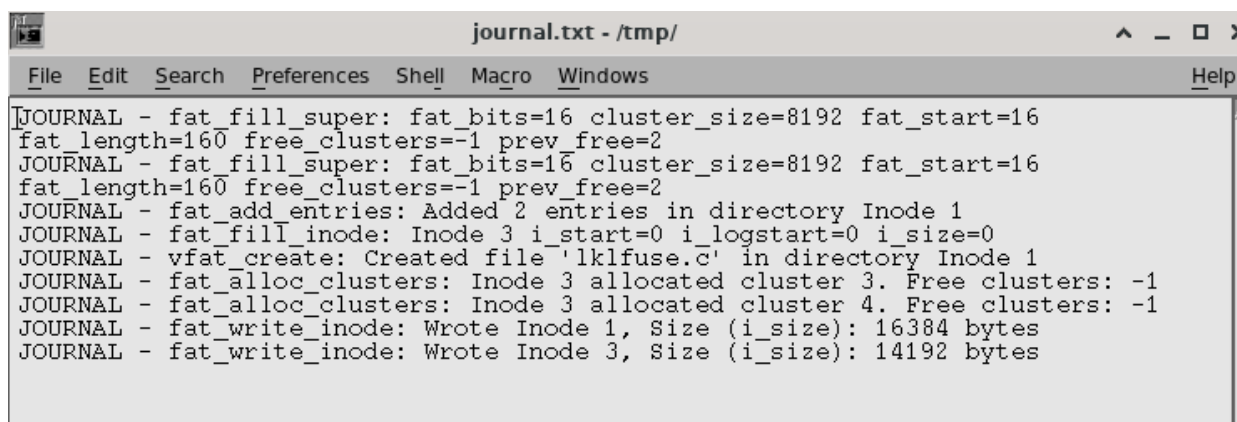
```
[ 0.011265] Executing now: fat_write_end
[ 0.011269] Executing now: generic_file_write_iter
[ 0.011278] Executing now: fat_write_begin
[ 0.011288] Executing now: fat_write_end
[ 0.011298] Executing now: generic_file_write_iter
[ 0.011307] Executing now: fat_write_begin
[ 0.011316] Executing now: fat_ent_blocknr
[ 0.011324] Executing now: fat_ent_bread
[ 0.011332] Executing now: fat16_ent_set_prt
[ 0.011340] Executing now: fat16_ent_bread
[ 0.011348] Executing now: fat16_ent_put
[ 0.011360] JOURNAL - fat_alloc_clusters: Inode 3 allocated cluster 4. Free clusters: -1
[ 0.011376] Executing now: fat_ent_blocknr
[ 0.011382] Executing now: fat_ent_bread
[ 0.011390] Executing now: fat16_ent_set_prt
[ 0.011398] Executing now: fat16_ent_bread
[ 0.011406] Executing now: fat_ent_blocknr
[ 0.011414] Executing now: fat_ent_bread
[ 0.011422] Executing now: fat16_ent_set_prt
[ 0.011430] Executing now: fat16_ent_bread
[ 0.011438] Executing now: fat16_ent_put
[ 0.011447] Executing now: fat_ent_blocknr
[ 0.011455] Executing now: fat_ent_bread
[ 0.011463] Executing now: fat16_ent_set_prt
[ 0.011471] Executing now: fat16_ent_bread
[ 0.011481] Executing now: fat_write_end
[ 0.011491] Executing now: generic_file_write_iter
[ 0.011499] Executing now: fat_write_begin
[ 0.011509] Executing now: fat_write_end
[ 0.011524] Executing now: fat_setattr
[ 0.016950] Executing now: fat_file_release
[ 1.011838] Executing now: fat_statfs
[ 1.011886] Executing now: fat_writepages
[ 1.011944] JOURNAL - fat_write_inode: Wrote Inode 3, Size (i_size): 14192 bytes
[ 1.013138] reboot: Restarting system
```

```

[ 0.079462] Executing now: fat16_ent_bread
[ 0.079471] Executing now: fat16_ent_put
[ 0.079500] JOURNAL - fat_alloc_clusters: Inode 3 allocated cluster 4. Free clusters: -1
[ 0.079561] Executing now: fat_write_end
[ 0.079566] Executing now: generic_file_write_iter
[ 0.079593] Executing now: fat_write_begin
[ 0.079605] Executing now: fat_write_end
[ 0.079634] Executing now: generic_file_write_iter
[ 0.079643] Executing now: fat_write_begin
[ 0.079670] Executing now: fat_ent_blocknr
[ 0.079679] Executing now: fat_ent_bread
[ 0.079703] Executing now: fat16_ent_set_prt
[ 0.079713] Executing now: fat16_ent_bread
[ 0.079721] Executing now: fat16_ent_put
[ 0.079762] JOURNAL - fat_alloc_clusters: Inode 3 allocated cluster 5. Free clusters: -1
[ 0.079797] Executing now: fat_ent_blocknr
[ 0.079798] Executing now: fat_ent_bread
[ 0.079806] Executing now: fat16_ent_set_prt
[ 0.079832] Executing now: fat16_ent_bread
[ 0.079840] Executing now: fat_ent_blocknr
[ 0.079848] Executing now: fat_ent_bread
[ 0.079891] Executing now: fat16_ent_set_prt
[ 0.079900] Executing now: fat16_ent_bread
[ 0.079908] Executing now: fat16_ent_put
[ 0.079928] Executing now: fat_ent_blocknr
[ 0.079957] Executing now: fat_ent_bread
[ 0.079965] Executing now: fat16_ent_set_prt
[ 0.079973] Executing now: fat16_ent_bread
[ 0.080001] Executing now: fat_write_end
[ 0.080013] Executing now: generic_file_write_iter
[ 0.080040] Executing now: fat_write_begin
[ 0.080051] Executing now: fat_write_end
error while reading directory .: Bad file descriptor
[ 0.080077] Executing now: fat_setattr
[ 0.097029] Executing now: fat_file_release
[ 1.080448] Executing now: fat_statfs
[ 1.080573] Executing now: fat_writepages
[ 1.080666] JOURNAL - fat_write_inode: Wrote Inode 3, Size (i_size): 14192 bytes
[ 1.081729] reboot: Restarting system

```

Και το αρχείο Journal.txt:



```

journal.txt - /tmp/
File Edit Search Preferences Shell Macro Windows Help
JOURNAL - fat_fill_super: fat_bits=16 cluster_size=8192 fat_start=16
fat_length=160 free_clusters=-1 prev_free=2
JOURNAL - fat_fill_super: fat_bits=16 cluster_size=8192 fat_start=16
fat_length=160 free_clusters=-1 prev_free=2
JOURNAL - fat_add_entries: Added 2 entries in directory Inode 1
JOURNAL - fat_fill_inode: Inode 3 i_start=0 i_logstart=0 i_size=0
JOURNAL - vfat_create: Created file 'lklfuse.c' in directory Inode 1
JOURNAL - fat_alloc_clusters: Inode 3 allocated cluster 3. Free clusters: -1
JOURNAL - fat_alloc_clusters: Inode 3 allocated cluster 4. Free clusters: -1
JOURNAL - fat_write_inode: Wrote Inode 1, Size (i_size): 16384 bytes
JOURNAL - fat_write_inode: Wrote Inode 3, Size (i_size): 14192 bytes

```

Εάν το ξανατρέξουμε:

```
journal.txt - /tmp/
File Edit Search Preferences Shell Macro Windows Help
JOURNAL - fat_fill_super: fat_bits=16 cluster_size=8192 fat_start=16
fat_length=160 free_clusters=-1 prev_free=2
JOURNAL - fat_fill_super: fat_bits=16 cluster_size=8192 fat_start=16
fat_length=160 free_clusters=-1 prev_free=2
JOURNAL - fat_add_entries: Added 2 entries in directory Inode 1
JOURNAL - fat_fill_inode: Inode 3 i_start=0 i_logstart=0 i_size=0
JOURNAL - vfat_create: Created file 'lklfuse.c' in directory Inode 1
JOURNAL - fat_alloc_clusters: Inode 3 allocated cluster 3. Free clusters: -1
JOURNAL - fat_alloc_clusters: Inode 3 allocated cluster 4. Free clusters: -1
JOURNAL - fat_write_inode: Wrote Inode 1, Size (i_size): 16384 bytes
JOURNAL - fat_write_inode: Wrote Inode 3, Size (i_size): 14192 bytes
JOURNAL - fat_fill_super: fat_bits=16 cluster_size=8192 fat_start=16
fat_length=160 free_clusters=-1 prev_free=2
JOURNAL - vfat_lookup: Found 'lklfuse.c' in directory Inode 1
JOURNAL - fat_fill_inode: Inode 3 i_start=3 i_logstart=3 i_size=14192
JOURNAL - fat_free: i_start: 3, i_logstart: 3
JOURNAL - fat_free_clusters: Inode 3 freed cluster 4. Free clusters: -1
JOURNAL - fat_free_clusters: Inode 3 freed cluster 268435455. Free clusters: -1
JOURNAL - fat_alloc_clusters: Inode 3 allocated cluster 3. Free clusters: -1
JOURNAL - fat_alloc_clusters: Inode 3 allocated cluster 4. Free clusters: -1
JOURNAL - fat_write_inode: Wrote Inode 3, Size (i_size): 14192 bytes
```

Παρατηρήσεις

Από το αρχείο journal μπορούμε να εξάγουμε τα εξής:

Κατά την πρώτη εκτέλεση του `./crtofs -p -i /tmp/disk -t vfat lklfuse.c` / το αρχείο `lklfuse.c` δεν υπάρχει στο σύστημα αρχείων FAT. Παρατηρούμε την αναμενόμενη ακολουθία: αρχικά καταγράφονται δύο φορές τα πεδία του superblock μέσω της `fat_fill_super` (ένα για κάθε mount), στη συνέχεια η `fat_add_entries` δεσμεύει 2 slots στον κατάλογο για το filename entry, η `fat_fill_inode` δείχνει `i_start=0` καθώς το inode μόλις δημιουργήθηκε χωρίς clusters, η `vfat_create` καταγράφει τη δημιουργία του αρχείου ενώ ακολουθούν δύο `fat_alloc_clusters` για τα clusters 3 και 4. Τέλος έχουμε δύο `fat_write_inode` για τον γονικό κατάλογο (Inode 1) και το αρχείο (Inode 3) με μέγεθος 14192 bytes.

Κατά τη δεύτερη εκτέλεση το αρχείο υπάρχει ήδη. Μετά το `fat_fill_super`, η `vfat_lookup` εντοπίζει το υπάρχον `lklfuse.c` στον κατάλογο (Inode 1), η `fat_fill_inode` το φορτώνει από τον δίσκο με `i_start=3 i_logstart=3 i_size=14192` επιβεβαιώνοντας ότι τα δεδομένα διαβάζονται σωστά. Στη συνέχεια η `fat_free` και οι `fat_free_clusters` αποδεσμεύουν τα παλιά clusters, ακολουθούν νέα `fat_alloc_clusters` και τέλος `fat_write_inode` με το ίδιο μέγεθος 14192 bytes, επιβεβαιώνοντας ότι το περιεχόμενο αντικαταστάθηκε επιτυχώς.

Υλοποίηση με lkl system calls

Δυστυχώς δεν καταφέραμε να χρησιμοποιήσουμε τα `lkl_sys` calls και παραμείναμε με τη χρήση της `extern`. Παρόλα αυτά θα δώσουμε μια σύντομη ιδέα για το πως πιστεύουμε πως θα υλοποιούνταν.

Αρχικά στο αρχείο `fat.h` θα έπρεπε να δηλωθεί μια μεταβλητή `int fd` στο struct `msdos_sb_info`. Έπειτα αντί για το `my_journal_fd` που δημιουργούμε εμείς θα

χρησιμοποιούσαμε τη μεταβλητή `sbi->fd` καθώς και θα αντικαθιστούσαμε τα `open` με `lkl_sys_open`, `close` με `lkl_sys_close`, `write` με `lkl_sys_write`.

Θεωρούμε πως κάπως έτσι θα έπρεπε να υλοποιηθεί παρόλο που σε εμάς δεν λειτούργησε.

